# CLOUD AUTOMATION SCRIPTING PROJECT REPORT

AWS Infrastructure Automation using Python, Bash Scripting, Cron Jobs, Monitoring, and Git Version Control


## Abstract

This project demonstrates cloud automation using Amazon Web Services (AWS), Python scripting, Bash scripting, cron job scheduling, and Git version control. The main objective of the project was to automate common cloud infrastructure and operational tasks in order to reduce manual effort, improve consistency, and increase efficiency.

Several AWS services were integrated in the project, including Amazon EC2 for virtual server provisioning, Amazon S3 for cloud storage, AWS IAM for identity and access management, Amazon CloudWatch for monitoring, and Amazon SNS for notifications. These services were managed programmatically using Python scripts and the boto3 SDK.

A Bash script was also developed to collect Linux system information such as disk usage, memory utilization, and running processes. The script was automated using cron scheduling to execute every five minutes. Git and GitHub were used for version control and project management.

The project provided practical experience in cloud automation, monitoring, operational scripting, and DevOps-oriented workflows. The implementation demonstrates how scripting-based automation can simplify and improve cloud infrastructure management.


## 1. Introduction

Cloud computing has become a fundamental part of modern IT infrastructure because it provides scalable, reliable, and on-demand computing services. Among the available cloud platforms, Amazon Web Services (AWS) is one of the most widely used for infrastructure management, storage, monitoring, and application deployment.

As cloud environments grow, managing resources manually becomes difficult, repetitive, and time-consuming. Cloud automation helps solve this problem by using scripts and APIs to automate cloud operations efficiently and consistently.

This project was developed to implement practical cloud automation using AWS, Python scripting, Bash scripting, cron scheduling, and Git version control. The project automates tasks such as EC2 provisioning, S3 bucket creation, IAM user management, CloudWatch monitoring, and SNS notifications using Python scripts and the boto3 SDK.

A Bash script was also developed for Linux system monitoring, and cron jobs were configured to automate periodic execution. Logging and exception handling were implemented to improve reliability and troubleshooting.

Overall, the project provided practical experience in cloud automation, monitoring, Linux scripting, and DevOps-oriented workflows.

## 2. Project Objectives

The primary objective of this project was to design and implement a practical cloud automation environment using AWS services and scripting technologies. The project focused on replacing manual cloud management tasks with automated workflows using Python, Bash scripting, and AWS APIs.

The key objectives of the project are:

- To understand and use the boto3 SDK for interacting programmatically with AWS services
- To automate Amazon EC2 instance provisioning using Python scripts
- To automate Amazon S3 bucket creation and file backup operations
- To automate AWS IAM user creation for identity and access management
- To configure Amazon CloudWatch alarms for infrastructure monitoring
- To implement Amazon SNS notifications for automated alert delivery
- To develop a Bash script for Linux system monitoring and log generation
- To schedule automated tasks using Linux cron jobs
- To implement logging and exception handling for reliable automation
- To manage project source code using Git and GitHub version control

In addition to automation, the project also aimed to provide practical exposure to important DevOps and cloud engineering concepts such as infrastructure automation, operational monitoring, scheduling, troubleshooting, and source code management.

## 3. Technologies Used

This project used multiple technologies and tools to implement cloud automation, monitoring, scheduling, and version control. Each technology played an important role in the overall automation workflow.

### 3.1 Programming and Scripting Languages

#### Python

Python was used as the primary automation scripting language because of its simplicity, readability, and strong integration with cloud platforms. Python scripts were developed to automate AWS services such as EC2, S3, IAM, CloudWatch, and SNS using the boto3 SDK.

### Bash Scripting

Bash scripting was used for Linux operational automation. A Bash script was developed to collect system-level information such as disk usage, memory utilization, and running processes, and store the outputs in log files.

## 3.2 AWS SDK and AWS CLI

### boto3 SDK

boto3 is the official AWS SDK for Python that enables Python scripts to communicate directly with AWS APIs. It was used throughout the project to automate AWS resource provisioning and monitoring tasks.

### AWS CLI

AWS Command Line Interface (CLI) was used to configure AWS authentication credentials and region settings inside the Ubuntu EC2 instance. The AWS CLI configuration was also used by boto3 for secure API access.

## 3.3 Version Control

### Git

Git was used for version control and project tracking. It helped in managing source code changes and maintaining project history.

### GitHub

GitHub was used as the remote repository platform for storing, managing, and sharing the project online.

## 3.4 Summary of Technologies

| Technology | Version / Type | Purpose in Project |
|---|---|---|
| Python | 3.x | Primary automation scripting language |
| boto3 SDK | Latest Stable Version | AWS SDK for Python used for AWS API integration |
| AWS CLI | Version 2 | AWS authentication and command-line management |
| Bash Scripting | GNU Bash | Linux system automation and monitoring |
| Cron | Linux Cron Daemon | Automated task scheduling |
| Git | Latest Stable Version | Local version control and source code tracking |
| GitHub | Cloud Hosted Platform | Remote repository management and code sharing |
| Ubuntu Linux | 22.04 LTS | Operating system used for the automation server |

## 4. AWS Services Used

This project used several core AWS services to implement cloud automation, monitoring, storage management, and notification handling. Each service was integrated programmatically using Python scripts and the boto3 SDK.

### 4.1 Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 is the compute service of AWS that provides virtual machines called instances. In this project, an Ubuntu-based t3.micro EC2 instance was provisioned automatically using a Python automation script instead of manual configuration through the AWS Console.

### 4.2 Amazon S3 (Simple Storage Service)

Amazon S3 is the object storage service of AWS used for storing files and backups. In this project, an S3 bucket was created programmatically, and file upload automation was implemented to simulate cloud backup operations.

### 4.3 AWS IAM (Identity and Access Management)

AWS IAM is used for authentication and access control in AWS environments. It allows administrators to manage users, roles, and permissions securely. In this project, IAM user creation was automated using Python scripting.

### 4.4 Amazon CloudWatch

Amazon CloudWatch is the monitoring and observability service of AWS. It collects infrastructure metrics and supports alarm generation based on defined thresholds. In this project, a CloudWatch alarm was configured to monitor EC2 CPU utilization automatically.

### 4.5 Amazon SNS (Simple Notification Service)

Amazon SNS is the notification and messaging service of AWS. It supports email, SMS, and other notification methods. In this project, an SNS topic and email subscription were configured to support automated alert notifications.

### 4.6 Summary of AWS Services

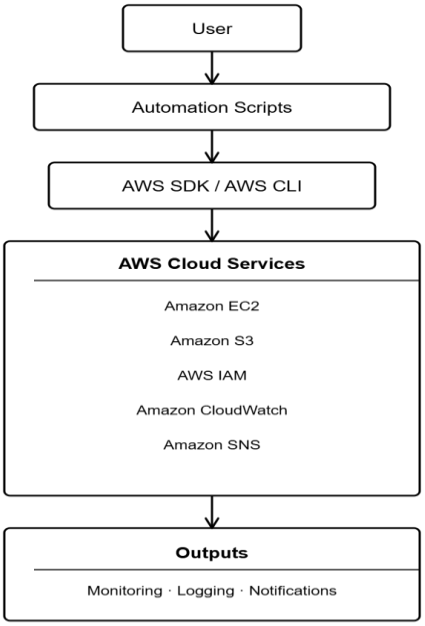
| AWS Service | Category | Purpose in Project |
|---|---|---|
| Amazon EC2 | Compute | Automated provisioning of virtual machine instance |
| Amazon S3 | Storage | Automated bucket creation and file backup upload |
| AWS IAM | Security | Automated IAM user creation |
| Amazon CloudWatch | Monitoring | Automated CPU utilization alarm configuration |
| Amazon SNS | Messaging | Automated notification and email subscription setup |

## 5. Project Architecture

The project architecture was designed to integrate automation scripts with AWS cloud services using Python, Bash scripting, AWS SDK, and Linux scheduling mechanisms.

The architecture follows a layered automation workflow where Python and Bash scripts interact with AWS services and Linux operating system components to automate infrastructure provisioning, monitoring, scheduling, and logging.

Architecture Flow:




## 5.1 Python Automation Layer

The Python automation layer acts as the primary cloud provisioning and management component of the project.

Python scripts were developed using boto3 SDK to communicate with AWS APIs programmatically.

This layer automates:

- EC2 provisioning
- S3 bucket creation
- File backup uploads

- IAM user creation
- CloudWatch alarm configuration
- SNS notification setup

Each Python script was implemented using exception handling and logging mechanisms to improve reliability and debugging.

## 5.2 Bash Automation Layer

The Bash automation layer was used for Linux operational automation.

A Bash script named system_info.sh was created to collect system monitoring information directly from the Ubuntu operating system.

The script collects:

- Current date and time
- Disk utilization
- Memory utilization
- Running process information

The collected information is stored inside log files for monitoring and operational analysis.

## 5.3 Monitoring Layer

Amazon CloudWatch was used as the infrastructure monitoring and observability service.

CloudWatch continuously monitors EC2 CPU utilization metrics and changes alarm states whenever the defined threshold value is crossed.

Monitoring automation helps administrators identify abnormal resource utilization conditions and infrastructure issues.

## 5.4 Notification Layer

Amazon SNS was integrated with CloudWatch alarms for automated notification delivery.

SNS topics and email subscriptions were configured so that monitoring alerts can be delivered automatically to subscribed users.

This layer demonstrates event-driven notification automation.

## 5.5 Scheduling Layer

Linux cron scheduling was implemented to automate recurring operational tasks.

The Bash monitoring script was configured to execute automatically every five minutes using cron expressions.

This scheduling mechanism eliminated the need for manual execution of monitoring scripts.

### 5.6 Version Control Layer

Git and GitHub were used as the version control and repository management layer.

Git tracked all project changes and maintained commit history, while GitHub stored the remote repository for backup and project management.

## 6. Project Workflow

The project implementation followed the below workflow:

1. Configure AWS CLI inside Ubuntu EC2 instance
2. Install boto3 SDK and required packages
3. Develop Python automation scripts
4. Automate EC2 provisioning
5. Automate S3 bucket creation
6. Upload files to S3 bucket
7. Create IAM users using Python
8. Configure CloudWatch CPU alarms
9. Configure SNS notifications
10. Develop Bash monitoring script
11. Configure cron scheduling
12. Generate logs and outputs
13. Push project to GitHub using Git

## 7. AWS CLI Configuration

Before running the automation scripts, AWS CLI was configured using IAM credentials.

Command Used:

aws configure

Configuration Details:

| Parameter | Value |
| --- | --- |
| Region | us-east-2 |
| Output Format | json |

AWS CLI configuration was necessary because boto3 uses these credentials to authenticate AWS API requests.

# 8. Python Automation Scripts

## 8.1 EC2 Provisioning Script

File Name: create_ec2.py

Purpose: Automatically launches an EC2 instance using boto3 SDK.

Technical Details:

| Parameter | Value |
| --- | --- |
| Region | us-east-2 |
| Instance Type | t3.micro |
| Operating System | Ubuntu Linux |

Features:

- Automated EC2 provisioning
- Exception handling using try-except
- Logging support

Output: EC2 Instance Created Successfully

## 8.2 S3 Bucket Automation Script

File Name: create_s3.py

Purpose: Automatically creates an S3 bucket.

Features:

- Region-specific bucket creation
- Storage automation
- Error handling

Output: S3 Bucket Created Successfully

## 8.3 File Backup Script

File Name: backup_s3.py

Purpose: Uploads files automatically to the S3 bucket.

Features:

- File upload automation
- Cloud backup implementation
- S3 integration

Output: Backup Uploaded Successfully

## 8.4 IAM Automation Script

File Name: create_iam_user.py

Purpose: Creates IAM users programmatically.

Features:

- IAM automation
- Security management
- Exception handling

Output: IAM User Created Successfully

## 8.5 CloudWatch Alarm Script

File Name: create_alarm.py

Purpose: Creates CloudWatch CPU utilization alarm for EC2 monitoring.

Alarm Configuration:

| Parameter | Value |
|---|---|
| Metric | CPUUtilization |
| Threshold | 70% |
| Evaluation Period | 1 |
| Monitoring Service | CloudWatch |

Features:

- Infrastructure monitoring
- CPU usage monitoring
- Alarm automation

Output: CloudWatch Alarm Created Successfully

## 8.6 SNS Notification Script

File Name: create_sns.py

Purpose: Creates SNS topic and email subscription.

Features:

- Notification automation
- Email alerts
- SNS topic creation

Output: SNS Topic and Email Subscription Created

## 9. Bash Monitoring Script

File Name: system_info.sh

Purpose: Collects Linux system information automatically.

Information Collected:

- Date and time
- Disk usage
- Memory usage
- Running processes

Linux Commands Used:

| Command | Purpose |
|---|---|
| date | Displays current date and time |
| df-h | Displays disk usage |
| free-h | Displays memory usage |
| ps aux | Displays running processes |

The Bash script stores the outputs into log files for monitoring and troubleshooting.

## 10. Cron Job Scheduling

Cron was used to automate periodic execution of the Bash monitoring script.

Cron Configuration:

*/5 * * * * /home/ubuntu/cloud-automation-project/bash-scripts/system_info.sh

Meaning: The script runs automatically every 5 minutes.

Benefits:

- Automated task execution
- Continuous monitoring
- Reduced manual effort

Cron execution was verified using log timestamps and Ubuntu cron logs.

## 11. Logging and Error Handling

Logging and exception handling were implemented in all automation scripts.

Python try-except blocks were used to handle runtime errors gracefully.

Generated Log Files:

| Log File | Purpose |
|---|---|
| ec2.log | EC2 instance provisioning logs |
| s3.log | S3 bucket creation and storage operation logs |
| iam.log | IAM user creation and access management logs |
| cron_log.txt | Cron job execution logs |
| system_info.txt | Linux system monitoring outputs |

Benefits:

- Easier troubleshooting
- Execution tracking
- Improved reliability

## 12. Git Version Control

Git was used for project version control and source code management.

Git Commands Used:

| Command | Purpose |
|---|---|
| git init | Initialize repository |
| git add . | Add project files |
| git commit | Save project version |
| git push | Upload project to GitHub |

GitHub Repository:

https://github.com/sujankhanofficial/cloud-automation-project

Benefits:

- Project tracking
- Remote backup
- Version management

## 13. Challenges Faced

| Challenge | Solution |
|---|---|
| AWS credential issue | Configured AWS CLI correctly |
| S3 region error | Added LocationConstraint parameter |
| CloudWatch alarm testing | Temporarily lowered threshold |
| GitHub authentication issue | Used Personal Access Token |
| Cron file path issue | Used absolute file paths |

## 14. Learning Outcomes

Through this project, practical knowledge was gained in:

- AWS cloud services
- Python automation scripting
- boto3 SDK usage

- Linux Bash scripting
- Cron scheduling
- Monitoring and observability
- Logging and troubleshooting
- Git and GitHub version control
- DevOps automation concepts

## 15. Conclusion

This project successfully demonstrates practical cloud automation using AWS cloud services, Python scripting, Bash scripting, cron scheduling, monitoring, logging, and Git version control.

The project automated multiple cloud infrastructure and operational tasks that are commonly performed manually in cloud environments.

Using Python scripting and boto3 SDK, several AWS services were integrated and automated successfully, including:

- Amazon EC2 for virtual machine provisioning
- Amazon S3 for cloud storage and backup automation
- AWS IAM for identity and access management
- Amazon CloudWatch for infrastructure monitoring
- Amazon SNS for automated notifications

The implementation showed how AWS APIs can be accessed programmatically to provision and manage cloud resources without depending completely on the AWS Management Console.

Linux operational automation was implemented using Bash scripting. The Bash monitoring script collected system-level information such as disk usage, memory usage, timestamps, and running process information.

Cron scheduling automated the periodic execution of the monitoring script every five minutes, demonstrating how recurring operational tasks can be automated efficiently.

Logging and exception handling mechanisms improved the reliability and maintainability of the automation scripts. Log files generated during execution helped in monitoring operations and troubleshooting runtime issues.

Git and GitHub were used for source code management and version tracking. The project repository was successfully uploaded to GitHub, demonstrating proper version control practices.

Through this project, practical understanding was gained in:

- Cloud infrastructure automation
- AWS service integration
- Python scripting using boto3
- Linux administration and Bash scripting
- Monitoring and observability concepts
- Cron-based task scheduling
- Logging and troubleshooting
- Git version control workflows
- DevOps-oriented automation practices

The project provided valuable hands-on experience with real-world cloud automation concepts and demonstrated how scripting technologies can simplify infrastructure management and operational activities in modern cloud environments.

The implementation also highlighted the importance of automation in improving operational efficiency, reducing manual effort, increasing consistency, and supporting scalable cloud infrastructure management.

Overall, all project objectives and deliverables were completed successfully, and the project achieved its goal of implementing a functional cloud automation environment using AWS and scripting technologies.